Project Synopsis — Submission Template

HCLTech Career Shaper Programme | III Year Industry Project | 2025-26

Field	Details (Learner to complete)
Project Title	Loan Default Prediction System: Explainable Credit Risk Scoring with Drift Monitoring
Project No.	137
Domain	Data Analytics
Difficulty	Intermediate
Duration	4 Weeks
Format	Individual Project
Learner Name	Lakshay [SURNAME]
Roll Number	[ROLL NUMBER]
Admission Number	[ADMISSION NUMBER]
Department / Class	Computer Science and Engineering (CSE), Class [CLASS]
College / Campus	[COLLEGE, CITY]
Mentor Name	[MENTOR NAME]
Submission Date	01 November 2026

1. Problem Statement

Who: credit officers and risk managers at a bank or lending company that approves personal loans.

What: approval depends on the lender's loan grade alone (grades A to C are approved), so two applicants with the same grade get the same answer whatever their income or loan size, and a declined applicant gets no reason; in the data, 21.8% of the loans defaulted.

When / where: at every loan application, and again later, because the mix of applicants shifts when the economy changes.

Why it matters: defaulted loans (non-performing assets, NPA) cost the lender money, and the RBI Fair Practices Code expects reasons for a rejection.

How: predict each applicant's probability of default with a machine learning model, turn it into APPROVE, REVIEW or REJECT with up to three plain-language reasons (SHAP), serve it through an API that logs every decision, and watch for drift in the applicants.

2. Proposed Solution & Approach

A Python pipeline (pandas, scikit-learn, XGBoost) cleans the public Kaggle Credit Risk Dataset (32,581 rows to 32,409), splits it 70/15/15 (stratified, seed 42), trains logistic regression, random forest and XGBoost, and keeps the model with the best validation AUC (XGBoost). SHAP turns each score into up to three reasons a loan officer can read out. A FastAPI service scores one application at a time (POST /predict) and offers /health, /model/info and /monitoring/drift; every scored application is written to a SQLite log. The decision policy is APPROVE below PD 0.20, REVIEW from 0.20 to 0.50 and REJECT from 0.50. Drift is measured with the population stability index (PSI) and an Evidently HTML report, and the kill switch SCORING_DISABLED=1 turns every decision into REVIEW while still scoring and logging. The system runs locally on Windows; it has no web front end and no login.

Demo: Runs locally; demo through Swagger UI at http://127.0.0.1:8000/docs

GitHub repository: [GITHUB REPOSITORY URL]

3. System Architecture

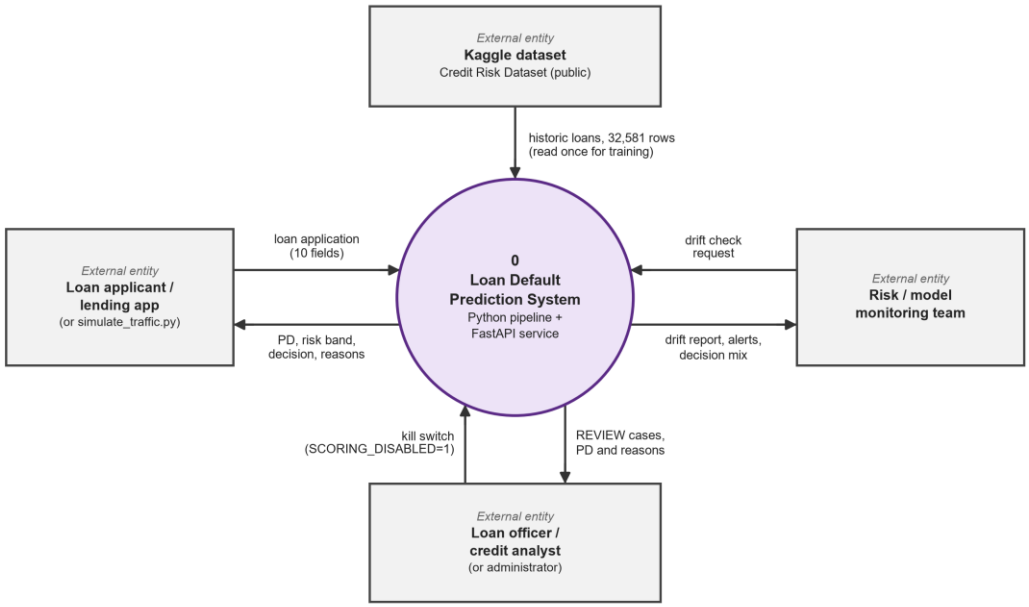


Figure 1: Data flow diagram, level 0 (Loan Default Prediction System and its external entities).

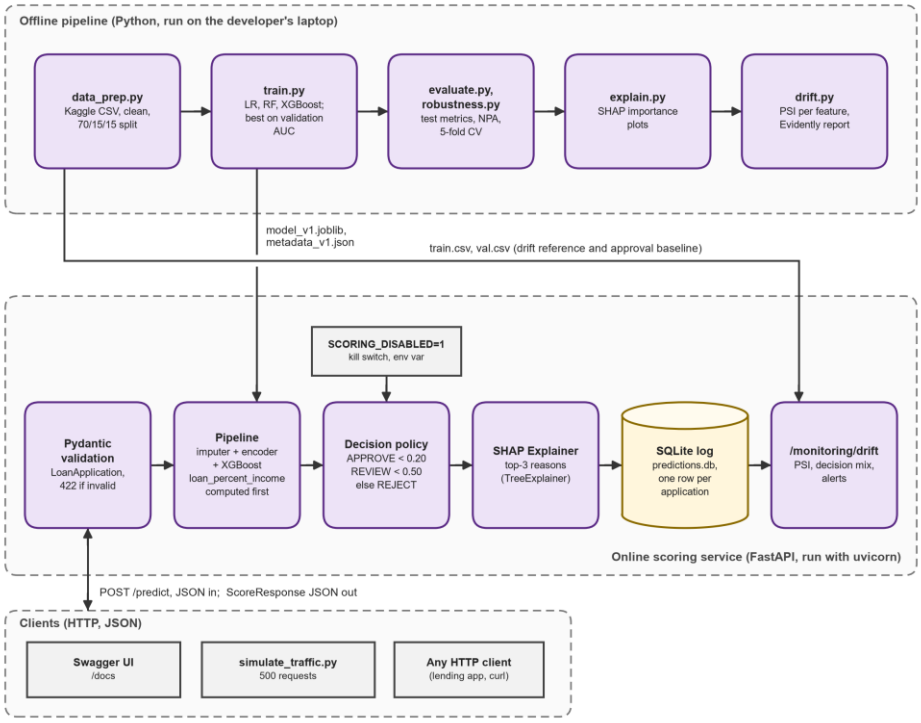


Figure 2: System block diagram (offline Python pipeline and the FastAPI scoring service).

4. Technology Stack

Frontend / UI:	No custom web front end. The Swagger UI that FastAPI serves at /docs is the demo screen.
Backend / Logic:	Python 3.13, FastAPI 0.141 on the uvicorn server, Pydantic 2.13 for input validation, PSI drift check, kill switch (SCORING_DISABLED).
Database:	SQLite file logs/predictions.db, one row per scored application.
AI/ML Framework:	pandas 3.0, scikit-learn 1.8 (logistic regression, random forest), XGBoost 3.2 (deployed model), SHAP 0.51 (reasons), Evidently 0.7 (drift report).
Visualisation:	matplotlib and seaborn (EDA and result charts), Evidently HTML report, Swagger UI.
Hardware:	A Windows laptop; everything runs on the CPU. No GPU, no cloud.

5. Dataset Used

Dataset file name:	credit_risk_dataset.csv (Kaggle "Credit Risk Dataset" by laotse, kept unchanged in data/raw/)
Number of rows:	32,581 raw; 32,409 after cleaning (172 removed: 165 exact duplicates, 5 ages over 100, 2 employment lengths over 60 years)
Number of columns:	12
Key columns (list 5):	loan_status (target, 1 = defaulted), loan_grade, loan_int_rate, loan_percent_income, person_income
Any additional data collected:	None. The file is used as downloaded; no personal identifiers are in it.

The records are simulated credit-bureau data, not a real lender's book. The default rate of 21.8% is far above the roughly 1.8% gross NPA ratio of Indian banks, so absolute NPA numbers do not transfer to a real lender; the relative gain between the grade rule and the model is the useful result.

6. Key Results & Metrics

All figures are from the 4,862-row held-out test set (stratified 70/15/15 split, seed 42).

Metric 1: Test ROC-AUC (XGBoost, deployed): 0.9469, against a target of at least 0.85. Random forest scores 0.9291 and logistic regression 0.8614. 5-fold cross-validation gives 0.9497 with a standard deviation of 0.0024.

Metric 2: NPA cut against the grade A-C rule: approving the same 85.5% of applicants by lowest model PD lowers the NPA ratio (defaulted share of the money lent) from 17.75% to 8.12%, which is 54.3% lower (target: at least 30%).

Metric 3: Decision bands: APPROVE covers 75.3% of applicants and 5.0% of them default; REVIEW covers 8.3% (26.1% default); REJECT covers 16.4% (97.0% default).

Response time of the app: 95th percentile of POST /predict is 45.6 ms over 500 requests (target: under 100 ms).

Business value achieved: simulated downturn traffic (incomes down 20%, rates up 2 points) raises two drift alerts (loan_int_rate PSI 1.178 and decision_mix), while normal traffic raises none. All 1,201 REVIEW and REJECT cases in the test set carry at least one reason. The 13 automated tests pass.

7. Challenges Faced & How Resolved

Challenge 1: XGBoost results change slightly with the number of CPU threads, so the numbers given in the project specification could not be reproduced exactly (test AUC 0.9465 to 0.9478 across 1 to 16 threads).

→ **Resolution:** measured the spread, kept the specified parameters, and report this project's own numbers (0.9469) together with the range; every decision, band and reason order still matches.

Challenge 2: Evidently 0.7 changed its API; the older evidently.report and evidently.metric_preset modules no longer exist, so code written for the older versions does not run.
→ **Resolution:** rewrote the report code with the 0.7 classes (Report, DataDriftPreset, Dataset.from_pandas, DataDefinition) and saved the HTML report to reports/drift_report_downturn.html.

Challenge 3: On Windows an open SQLite connection keeps the log file locked, and Python's with on a connection commits but does not close it.
→ **Resolution:** every database access goes through one helper that always closes the connection, and the tests write to a temporary database so they never touch the real log.

8. References

1. Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785-794.
2. Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems 30*, 4765-4774.
3. Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830. https://jmlr.org/papers/v12/pedregosa11a.html
4. Siddiqi, N. (2017). *Intelligent Credit Scoring: Building and Implementing Better Credit Risk Scorecards* (2nd ed.). Wiley.
5. Government of India. *The Digital Personal Data Protection Act, 2023* (Act No. 22 of 2023).

9. Ethical Compliance Declaration

I confirm that:

☑ All dataset usage complies with the source licence.

☑ No personally identifiable information (PII) is stored without consent.

☑ AI tools used for code/writing are disclosed in the Appendix (Claude by Anthropic).

☑ This project complies with the DPDP Act 2023 where applicable.

Learner Signature: Lakshay [SURNAME] Date: 01 November 2026